

객체지향프로그래밍

assignment 2

26.09.13

시스템생물학과 2023161036 박수영

1. Design considerations and choices of programming constructs

1) 소수 판독기

main 함수 호출 후 identifier로 number와 임의의 정수 i를 설정한다.

number와 i는 int type으로 선언한다.

먼저 cout으로 "Enter a number between 2 and 1,000 (inclusive):" 를 출력한 후, cin으로 number를 입력받는다.

i를 2부터 (number-1)까지 증가시키며 나눠서 나누어떨어지는지 확인한다. 따라서 for 문을 사용하여 반복하도록 한다.

if 문과 연산자 %를 사용하여 나누어 떨어지는지 확인하고, 나누어 떨어진다면(number % i == 0) "number is not a prime."을 출력한다.

그렇지 않다면 "number is a prime."을 출력한다.

2) 소수 출력기

main 함수 호출 후 identifier로 number와 임의의 정수 i, j를 설정한다.

number와 i, j는 int type으로 선언한다.

먼저 cout으로 "Enter a number between 2 and 1,000 (inclusive):" 를 출력한 후, cin으로 number를 입력받는다.

loop1: j가 2부터 number 이하일 때까지 1씩 증가시키며 판독할 숫자를 계속 바꾼다.

loop2: i를 2부터 j-1까지 1씩 증가시키며 number % i를 실행하여 소수인지 검사한다. 소수일 때마다 cout 을 사용하여 j를 출력한다.

2. Source Listing

1) 소수 판독기

```
#include <iostream>
using namespace std;

int main() {

    int number;
    int i;

    cout << "Enter a number between 2 and 1,000 (inclusive): ";
    cin >> number;

    for (i = 2; i < number; i++) {
        /*i를 2부터 (number-1)까지 증가시키며 나누어 떨어지는지 확인하기*/
        if (number % i == 0) {
            cout << number << " is not a prime.\n";
            return 0;
        }
    }

    cout << number << " is a prime.\n";

    return 0;
}
```

2) 소수 출력기

```
#include <iostream>
using namespace std;

int main() {

    int number;
    int i;
    int j;

    cout << "Enter a number between 2 and 1,000 (inclusive): ";
    cin >> number;

    for (j = 2; j <= number; j++) {
/* 숫자를 하나씩 바꾸며 j가 소수인지 검사한다 */
        for (i = 2; i < j; i++) {
            if (j % i == 0) {
                break;
            }
        }

        if (i == j) {
            cout << j << endl; //소수라면 j를 그대로 출력한다
        }
    }

    return 0;
}
```

3. Test results

1) 소수 판독기

```
suyeong@suyeongui-MacBookAir OOP % /Users/suyeong/OOP/prime_number
```

```
Enter a number between 2 and 1,000 (inclusive): 4
```

```
4 is not a prime.
```

```
suyeong@suyeongui-MacBookAir OOP % /Users/suyeong/OOP/prime_number
```

```
Enter a number between 2 and 1,000 (inclusive): 2
```

```
2 is a prime.
```

```
suyeong@suyeongui-MacBookAir OOP % /Users/suyeong/OOP/prime_number
```

```
Enter a number between 2 and 1,000 (inclusive): 1000
```

```
1000 is not a prime.
```

```
suyeong@suyeongui-MacBookAir OOP % /Users/suyeong/OOP/prime_number
```

```
Enter a number between 2 and 1,000 (inclusive): 49
```

```
49 is not a prime.
```

```
suyeong@suyeongui-MacBookAir OOP % /Users/suyeong/OOP/prime_number
```

```
Enter a number between 2 and 1,000 (inclusive): 5
```

```
5 is a prime.
```

2) 소수 출력기

```
suyeong@suyeongui-MacBookAir OOP % /Users/suyeong/OOP/prime_number2
```

```
Enter a number between 2 and 1,000 (inclusive): 23
```

```
2
```

```
3
```

```
5
```

```
7
```

```
11
```

```
13
```

```
17
```

19

23

suyeong@suyeongui-MacBookAir OOP % /Users/suyeong/OOP/prime_number2

Enter a number between 2 and 1,000 (inclusive): 11

2

3

5

7

11

suyeong@suyeongui-MacBookAir OOP % /Users/suyeong/OOP/prime_number2

Enter a number between 2 and 1,000 (inclusive): 6

2

3

5

suyeong@suyeongui-MacBookAir OOP % /Users/suyeong/OOP/prime_number2

Enter a number between 2 and 1,000 (inclusive): 13

2

3

5

7

11

13

3. Discussion

현재 프로그램은 loop를 중첩하여 사용해서 각 숫자가 소수인지 하나씩 확인하는 방식이다. 더 정리된 프로그램을 만들기 위해서는 소수를 판별하는 부분을 함수로 묶어 사용할 수 있을 것이다. 그렇게 한다면 소수 판독이 필요할 때마다 불러와서 반복적으로 사용할 수 있다.

또한 이미 찾은 소수들을 저장해 놓는 데이터 모음이 있다면 매번 계산할 필요 없이 불러오기만 하면 되므로 더 효율적으로 프로그램을 동작시킬 수 있을 것이다. 그리고 이 방식을 사용한다면, 에라토스테네스의 체를 사용하여 소수를 빠르게 찾을 수 있다.

그리고 실제로는 해당 숫자의 제곱근까지만 확인해도 소수인지 판단할 수 있으므로 이 방식으로 코드를 짤다면 loop의 반복 횟수를 줄일 수 있다.